

How the Classification Tree Works

44%

nested LOO-CV accuracy, leakage-free · vs. 38%
majority-class baseline

The tree predicts which tercile of the 252d parent CAR distribution an event lands in — **underperform** / **middle** / **outperform** — from four features: **prob_included**, **forced_flow_adv**, **log_child_mktcap**, **beta**. Selection and scoring both use nested LOO-CV so no decision ever sees the label it's predicting. $n = 16$ events.

HOW IT'S FIT: NESTED LOO-CV + A 3-LEAF FLOOR

LEAKAGE-FREE

for each held-out event (outer LOO):

grow full tree on the other 15 → candidate **ccp_alphas**

keep only alphas whose tree has ≥ 3 leaves

(a 2-leaf tree can't output a 3rd class — floor makes

"underperform" structurally reachable)

inner LOO over those 15 picks the best-scoring alpha

refit on all 15 → predict the held-out event

- Every outer fold picked **alpha=0** — at $n=16$, the ≥ 3 -leaf floor already excludes every more-pruned candidate, so the search confirms the full, unconstrained tree is the only option that qualifies.

MAJORITY BASELINE

38%

NESTED LOO ACCURACY

44%

- "Underperform" recall: **0%** in last week's 2-leaf tree → **40%** now — the floor is what makes flagging the downside case possible at all.

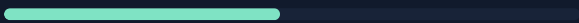
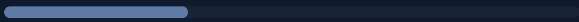
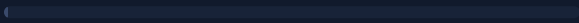
THE LEARNED TREE (FIT ON ALL 16 EVENTS)

DEPTH 3 · 6 LEAVES

```

├─ forced_flow_adv ≤ 7.43
│   └─ prob_included ≤ 0.62
│       └─ beta ≤ 0.98 → middle
│       └─ beta > 0.98 → underperform
│   └─ prob_included > 0.62 → middle
├─ forced_flow_adv > 7.43
│   └─ prob_included ≤ 0.36 → underperform
│       └─ prob_included > 0.36
│           └─ forced_flow_adv ≤ 18.35 → outperform
│           └─ forced_flow_adv > 18.35 → underperform

```

forced_flow_adv  .52
 prob_included  .29
 beta  .19
 log_child_mktcap  .00

	pred: under	pred: mid	pred: over
actual: under	2	1	2
actual: mid	2	3	1
actual: over	2	1	2

Nested LOO-CV predictions, all 16 events. Diagonal = correct. Forced-selling flow (**forced_flow_adv**) drives most splits — more than **prob_included** alone.

n=16

Read as hypothesis, not strategy. **44% vs. 38%** is a real, leakage-free improvement — but it comes from one tree shape confirmed across all 16 outer folds, not a genuine complexity search. Next: how this tree's predictions turn into a position-sizing weight (Slide 2 of 2).